Manipulación de animaciones

Arquitectura, navegación y animación

Objetivos de la clase

- Comprender el rol de la arquitectura en el desarrollo escalable de una app.
- Diseñar flujos de navegación coherentes y reutilizables.
- Identificar e implementar animaciones clave para reforzar la experiencia de usuario.
- Aplicar conocimientos previos en una estructura realista de proyecto.

1. Definición de alcance

1.1 ¿Cuál es el alcance de una app?

Es el conjunto de funcionalidades, pantallas, datos y comportamientos que estarán presentes en una aplicación. Antes de comenzar a programar, se debe definir qué se va a construir y qué no.

1.2 Elementos del alcance

Elemento	Ejemplo
Público objetivo	Usuarios que desean registrar hábitos de estudio.
Funciones clave	Registro, login, tracking de tareas, recordatorios.
Contenido principal	Tareas, sesiones, historial, estadísticas.
Navegación esperada	Inicio → Detalles → Crear → Perfil.
Accesibilidad offline	¿Se puede usar sin conexión? ¿Qué datos se guardan local?

1.3 Mapa de alcance simplificado

```
- Bienvenida / Onboarding
- Login / Registro
- Dashboard con Tabs
  - Lista de tareas (Stack)
  - Historial
  - Perfil
- Crear nueva tarea (modal o animación flotante)
```

1.4 Separación por módulos

Organizar el código en módulos o features ayuda a escalar el proyecto:

```
📁 src/
  ├── auth/
  ├── tasks/
  ├── profile/
  ├── navigation/
  ├── components/
  ├── styles/
```

2. Flujos de navegación

2.1 Tipos de flujo

Tipo	Descripción
Autenticación	Login → Registro → Recuperar contraseña
Onboarding	Pantallas introductorias con skip o continuar
App principal	Tabs, Drawers o una combinación con Stacks anidados
Deep linking (avanzado)	Acceder a una pantalla específica desde una notificación o link externo

2.2 Composición típica de navegadores

```
<NavigationContainer>
```

```
  <Stack.Navigator>
    {user ? (
      <Stack.Screen name="App" component={TabsNavigator} />
    ) : (
      <Stack.Screen name="Login" component={LoginScreen} />
    )}
  </Stack.Navigator>
</NavigationContainer>
```

Desglose del árbol

- **NavigationContainer**
  - Stack.Navigator
    - Splash *(mientras resolvemos sesión/rehidratamos estado)*
    - App *(TabsNavigator) (si user existe)*
      - Tab/Home -> HomeStack
      - Tab/Explore -> ExploreStack
      - Tab/Profile -> ProfileStack
    - Login *(si user === null)*
    - Modal *(capa modal global - opcional)*

Este árbol deja explícito qué pantalla “queda arriba” de la pila en cada estado.

Estado global	Descripción	Pantalla renderizada	Nota de UX/Animación
loading===true	Chequeo de token / AsyncStorage / refresh	**Splash**	Fade-in corto (300ms). Evitar parpadeo entre Splash y siguiente.
loading===false && user	Sesión válida	**App (TabsNavigator)**	Transición de Stack: fade o slide suave al entrar al App.
loading===false && !user	Sin sesión	**Login**	Presentación estándar; al loguear, reset() hacia App.

logout()	El usuario sale	Reseteo a **Login**	Usar CommonActions.reset() para limpiar historial.
openGlobalModal()	Acción contextual	**Modal** sobre la pantalla actual	presentation:'modal' con swipe-to-close si aplica.

2.3 Ejemplo de navegación compuesta

```
<Tab.Navigator>
  <Tab.Screen name="Tasks" component={TasksStack} />
  <Tab.Screen name="Stats" component={StatsScreen} />
  <Tab.Screen name="Profile" component={ProfileScreen} />
</Tab.Navigator>
```

Donde TasksStack podría ser:

```
<Stack.Navigator>
  <Stack.Screen name="TaskList" component={TaskListScreen} />
  <Stack.Screen name="TaskDetail" component={TaskDetailScreen}
/>
</Stack.Navigator>
```

2.4 Buenas prácticas

- Separar cada Navigator en su propio archivo.
- Usar constantes para los nombres de rutas.
- Organizar las rutas en un archivo central (routes.ts).
- Asegurarse de que las transiciones entre pantallas sigan una lógica narrativa (como un flujo de usuario).

3. Animaciones clave

3.1 ¿Por qué usar animaciones clave?

Las animaciones clave ayudan a:

- Reforzar cambios de estado.
- Aumentar la percepción de fluidez.
- Hacer más intuitiva la navegación.
- Guiar la atención del usuario.

3.2 Tipos de animaciones recomendadas

Tipo	Uso común
Fade in/out	Entrada y salida de componentes.
Slide (translateY/X)	Navegación entre pantallas, carruseles, drawers.
Escalado (`scale`)	Feedback en botones o cards.
Composición (`opacity + translate`)	Movimiento natural de componentes al cargarse.

3.3 Framer Motion: Revisión rápida

```
import { Motion } from '@legendapp/motion';

<Motion.View
  initial={{ opacity: 0, translateY: 50 }}
  animate={{ opacity: 1, translateY: 0 }}
```

```
  transition={{ duration: 400 }}
>
  <Text>¡Animado!</Text>
</Motion.View>
```

3.4 Microanimaciones y contexto UX

- Feedback instantáneo = mayor retención.
- Un shake() al fallar login guía al usuario.
- Un bounce() al completar una tarea refuerza positivamente.
- Animaciones largas no deben entorpecer la experiencia: mantener entre 200-600ms.



3.5 Transiciones entre pantallas

Podés usar @react-navigation/stack con opciones como:

```
<Stack.Screen
  name="Screen"
  component={MyScreen}
  options={{
    animation: 'slide_from_right', // Otros: fade, none, flip,
slide_from_bottom
  }}
/>
```

También podés crear transiciones personalizadas con ***cardStyleInterpolator***.

En esta clase, los alumnos comenzarán a:

- Definir el alcance real de su proyecto de entrega.
- Prototipar flujos completos con Stack + Tabs.
- Aplicar animaciones clave y microinteracciones a pantallas, botones, modales.



Bibliografía utilizada y sugerida

React Navigation – Concepts and Composition.

https://reactnavigation.org/docs/navigators

Actualizado en Julio 2025.

Framer Motion for React Native.

https://motion.legendapp.com/

Actualizado en Julio 2025.

UX Collective. When and why to use animations in mobile apps.

https://uxdesign.cc/using-animations-mobile-ux

Actualizado en Abril 2025.